

Year: 2026

Diary

Subj: Unisoft

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

MON
TUE
WED
THU
FRI
SAT
SUN

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☒ August ☐ September ☐ October ☐ November ☐ December

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☒ Thursday ☐ Friday

Document-1

Main Focus: AI coding agent use করে
Software Develop করা- ~~And~~ AI এর code creat
করাছে সেটা verify করা

AI coding Agents: ① Claude Code
② ZCode
③ VS Code Agent Mode

Document-2

AI-Agent Development Internship

* We hired for curiosity instead of experience

Agentie Engineering

ERP system is interconnected. So manual
checking is Difficult here AI can help us
But, if engineers blindly blive on Agent
its also dangerous.

Learning to tell these two situations apart
is, more or less, the whole job.

Year: Day - 1

Diary

Subj: Unisoft Overview.

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

MONTH

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

Unisoft Systems Ltd → Enterprise Software company

Product Family

- * ERP suites
- * Compliance and POS
- * Banking and Finance
- * Specialised System

Clients:

- ① City Bank, Nee, IFIE
- ② UEB, SouthEast Bank
- ③ MTB, BRAE Bank

NRB-approved software provider for VAT systems.

ISO 27001, SOC-2 certifications.

Year :

Diary

Subj : USLBD Product Review

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

2020

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

UniVAT

I Found this software Product really interesting because I had never come across an application before like this. I wasn't even aware that something like the existed, so that's what made me choose this product.

NBR-approved VAT Management System, which automate tax calculation, compliance reporting.

The purpose of UniVat is to automate and control the process of Vat.

UniVat can automatically calculate VAT over transactions.

- i) automatic vat calculation
- ii) multiple vat rate support
- iii) VAT exemption management
- iv) input tax credit calculation
- v) VAT adjustment/correction.

Multi-rate support etc

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

MONTH

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

1. What Problem does it solve?

Univat solves the problem of manually managing VAT calculation documentation, reporting and compliance. Business have many daily transactions and manually calculating vat report or preparing VAT reports can lead to error, delays and compliance issues. Univat automates these process and helps business manage their VAT-related activities and calculation accurately and more efficiently.

2. Who uses it every day?

Univat is mainly be used by finance and accounting teams, VAT/tax officers, business managers, and organization that need to manage VAT compliance. It can also received data from systems such as ERP, POS, accounting, banking and e-commerce platforms, so employees working with those systems can be indirectly connected to Univat.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

MON

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

3. What data does it hold?

Univat handles VAT and business transaction-related data.

- i) Sales and Purchase transaction
- ii) VAT amounts and VAT rates.
- iii) Input tax and adjustments.
- iv) Customer/supplier-related transaction information
- v) VAT returns and reports.
- vi) Mushak-related information.
- vii) Compliance and filing records.
- viii) Audit trails and transaction history.

In simple words it holds the information needed to calculate, track, report, and verify a company's VAT activities.

Year :

Diary

Subj : Day 2

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

2020
☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☒ August ☐ September ☐ October ☐ November ☐ December

DAY
☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☒ Friday

- i) Install Project Successfully.
- ii) Get Real project.

iii) Issue Detected.

Cloude Code is a paid Agent so I used VS code agent mode and /init command do not work there. So have to write CLAUDE.MD with Different command.

Analyze this entire project and understand its Architecture, structure, important files, development workflow and coding conventions. Do not modify any files.

Done.

iv) what this project Do. (3 sentence).

This project is a Flask Freamework itselfe. It provides the core application objects, routing, request handling, templates, configuration, and VETI tools that developer use to create web services. It is not a specific app; it is the freamework codebase that powers many Flax based projects.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Month

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

Day

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

2. (question num-2) where would I add a new command-line options?

Question → search → Find Relevant code →
Read → Decide where to modify.

● Agent Did before answering

i) searched for (regex). Regex is a search pattern. and find the CLI entry point and point the exact place.

ii) Finally it opened the file cli.py where the command line is located.

src / Flask / cli.py

iii) I am not able to detect the answer where the agent changed its mind. But will try again.

For top level flag, look around Flask Group.

Flask --app

Flask --debug

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Month

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

Day

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

For new option on the actual Flask run
look around the run command.

Flask run --port

3 what is the most confusing part of this
codebase for newcomers.

The most confusing part is the split
between the public app API and the
lower-levels `gens` - `io` internals. the
codebase mixes a very simple user-facing
interface with deeper registration systems
in `src/flask/sansio/scaffold.py` and the
context machinery in `src/flask/ctx.py`.

A newcomer may expect everything to
live in one place, but route hooks,
config, request context, and app setup
are intentionally spread across
several layers to support extensibility
and clean separation.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

2020

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

*which one of the three answers was best.

The first one is the best because it was directly grounded in the repo itself.

The third one was weakest. Because it was not directly supported by repository evidence and depended on speculation about newcomer experience rather than facts in codes.

Year :

Diary

Subj : Day-3

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

2020

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☒ August ☐ September ☐ October ☐ November ☐ December

☒ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

Plan for Milestone 1

1. Create a small python CLI project.
2. Add an entry point module using argparse.
3. Define subcommands:
add, list, search, delete.
4. Make each subcommand print exactly not implemented.
5. Add a minimal test file verifying all four commands.
6. Run the focused tests.

Milestone-2

The day-3 Project is intentionally minimal; test use `pytest`, imports `resolve` from the project root, and there is no existing storage format to preserve.

The likely local contract is JSON, with an empty file treated as no notes: I'll verify that against the milestone notes

Plan:

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

MON

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

TUE

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

1. Create Day-3/app/storage.py with:

i) load_notes(path): load JSON, return a list of note dictionaries, return [] for missing or empty files.

ii) save_notes(path, notes): write the notes list as JSON.

2. Add Day-3/tests/test_storage.py. Covering

i) Missing File

ii) Empty File

iii) One Note

iv) Round-tripping three notes.

Run the focused storage test with
pytest tests/test_storage.py.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Month

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

Day

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

LAB-2 (Explanation)

git diff --staged.

1. Storage.py

```
import json
from pathlib import Path
```

JSON Data पढ़ने का json
File का location/path निम्न काउ करके का Path

load_notes()

```
def load_notes(path):
    file_path = Path(path)
```

Function.

load_notes("notes.json")

जहाँ "notes.json" एक path.

Path(path) जहाँ file path दिखाए जा रहा है।

```
if not file_path.exists():
    return []
```

File empty है:

```
contents = file_path.read_text()
if not contents.strip():
    return []
```


Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

white space हटाकर
contents.strip()

File empty है तो
return []

JSON text → Python Data

return json.loads(contents)

दिए गए File को JSON read करें

def save_notes(path, notes):

path(path).write_text(json.dumps(notes))

Python notes को JSON बनाने वाली File में लिखें

Python notes



save_notes



JSON File



Load_notes()



Python notes.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Month
☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☒ August ☐ September ☐ October ☐ November ☐ December

Day
☒ Saturday ☒ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

NOTE Taking CLI

command : add
list
search
delete

This Note will save to a JSON file.

Milestone - 1

import argparse

the libra built-in module use for terminal
for command for user

COMMANDS = ("add", "list", "search", "delete")

It's a tuple. Only allowed command.

Year :

Diary

Subj : Day - 4

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Month

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

Day

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

Lab-3

Code passing test:

- i) Accept boundary discount
- ii) applies discount to multiple items
- iii) rejects discount above one hundred
- iv) reject discount below zero
- v) test rounds to two places using half up.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Month

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

Day

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

1. Empty [] items.

length 0, Falsy, Raises IndexError

Attempting to read from an empty list will crash your program.

(For) Loop over empty list skips the body entirely.

Absence of data.

~~Opinion: In this case empty list is invalid because if list is empty then the discount is not applied and billing system will not working.~~

If items is empty, the function returns Decimal(0.00). This happens because the subtotal at Decimal(0) and there are no items to add. This is the normal behavior for an empty invoice.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Month

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

Day

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

Quantity 0 or negative.

2. Quantity = 0

Current behavior: contribute 0 to subtotal.
concern: should zero quantity be allowed?

2. Quantity < 0

Creates a negative line amount.
should negative quantity be rejected?

A negative quantity can reduce the invoice total and may cause an incorrect bill

The function does not reject negative quantity.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

2020

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY
☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

3. Discount - percent

0% means no discount

100% discount mathematically correct.

4. Rounding application per item or to the total.

Rounds only the final discount total using ROUND-HALF-UP.

5. is Round-HALF-UP or banker's rounding.

Round-HALF-UP

The implementation uses Decimal for monetary calculations and does not use float.

7. Error message

No error message does not specify which message is wrong. It's only give value error with a message.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

M
O
N
T
H
☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December
D
A
Y
☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

Q-8: Do the test actually test the rejection

The existing boundary does not test the rejection. It only verifies that the valid boundary values 0% and 100% are accepted, and produce the expected totals. It does not test an out of range value or verify that a value error is raised.

Q-9: Rounding Boundary

There is no test for the rounding boundary.

Q-10: Converting Float.

No -

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

2020

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY
☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

Three most serious Issue.

- 1 Negative quantities are accepted
2. Rounding policy is not explicitly defined or tested.
- 3 Items values are not validated.

→ before fixing I was only applied on the subtotal but after fixing, the test expected per-item rounding.

Each item's discounted amount is now rounded to two decimals using ROUND_HALF_UP.
Before the items amounts are added.

The policy is also stated in the function docstring.

→ The function now raises `ValueError` when `quantity < 0`.

Year :

Diary

Subj : Day-5

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Month

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

Day

☐ Saturday ☐ Sunday ☐ Monday ☒ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

The Tools.

Model (brain): It reads and writes text.

i) GLM-5.2

ii) Kimi-3

iii) Deep Seek V4-Pro

iv) Claude Opus

The Harness (body): The ability to

i) read files ii) edit them

iii) run commands iv) see output

Example:

i) Claude Code

ii) ZCode

iii) Kimi Code

iv) VS Code agent mode

Most harness speak an Anthropic - compatible API, so switching the model is just two environment variable.

Year: 2020

Diary

Subj: LAB-4

Date: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

2020
☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

D
A
Y
☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

1. Empty Body: Potential bug. Application empty body accepted.

#2. Add note with 100000 character long.

Shell level: Argument is too long

Application level: Accepted

Bangla and Emgi → Successful

No existing delete → silently ignored.

#5. Special character → needs review. (ignored)

#6. Corrupted JSON file: Clean bug.

notes: json invalid এর application JSON code error
এর full traceback দিয়ে crash করে।

Regression: Corrupted JSON এর ক্ষেত্রে application
এর unhandled exception/traceback এ
দিয়ে graceful handle করে

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Month

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

Day

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

#7. Read only JSON

permissionError unhandled অবশ্যই full
traceback দিয়ে crash করাছে।

file write না করা হলে application যাবে
gracefull এবং permission handle করে

#8: Successfull.

Year :

Diary

Subj : UNIDESK

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

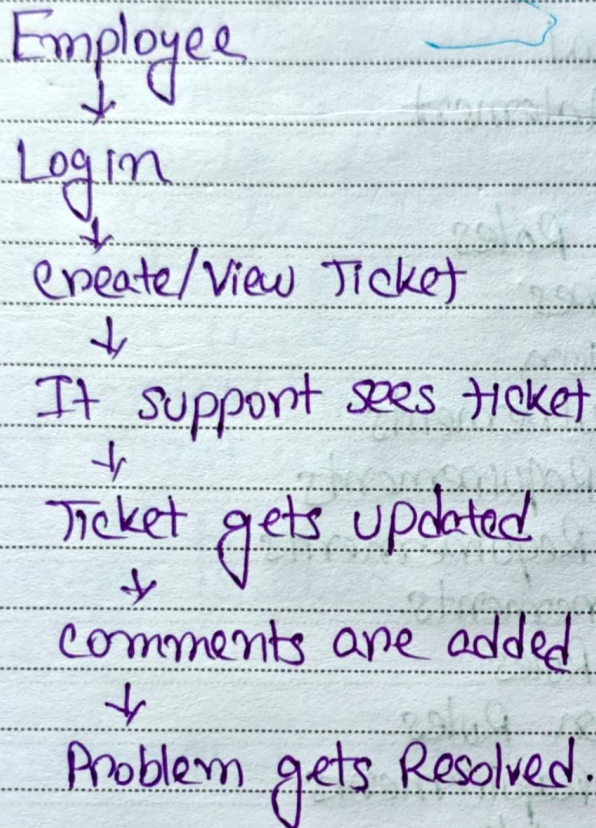
Month

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

Day

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

UniDesk - Mini Internal Ticketing System



Data

User

Tickets

Comments

PostgreSQL Database

Fast API

React Frontend

JWT authentication

Ticket CRUD

Input Validation

API test

70%+ API test coverage.

Year: 2020

Diary

Subj:

Date: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

2020
☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY
☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

Goal: Working ticketing system

- i) Project Goal
- ii) Problem statement
- iii) scope
- iv) Users and Roles
- v) Core Features
- vi) Authentication
- vii) Ticket Requirements
- viii) Comment Requirements
- ix) Database Requirements
- x) API Requirements.
- xi) Validation Rules
- xii) Autorization Rules
- xiii) Frontend Requirements
- xiv) Error Handling
- xv) Testing Requirements.
- xvi) Out of scope
- xvii) Defination of Done.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

MONTH

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

Defination of Done:

1. User can authenticate
2. Authorized user can create a ticket
3. User can view appropriate tickets.
4. Ticket can be updated.
5. Ticket can be deleted according to permission.
6. Comments work.
7. Validation works.
8. Unauthorized requests are rejected
9. React Client work with API
10. API test pass.
11. coverage is above 70%.

Alice Johnson → abedefgh2

0

→ Abedefgh2

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

MON
TUE
WED
THU
FRI
SAT
SUN

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

D
A
Y

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

* Large Module Integration.

Add Implement ticket Assignment and Agent work Queues.

Add supported-agent assignment so agent can claim tickets, view assigned work and release tickets.

* Large Architecture / security / complex logic

Harden Authentication and Session Management.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

MON
TUE
WED
THU
FRI
SAT
SUN
☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

1. Dashboard empty-state check

Dashboard view JSX.

For the tiny task 1, and it is a one-time UI task! so Agent.md did not make the coding itself significantly better. The change I was simple enough either way.

Without agent, Earlier task also caused confusion about migration, database state, and ambiguous lifecycle rules.

With agent.

- i) Inspect the existing implementation before editing
- ii) Check the request behavior already exists
- iii) Make the smallest possible change.
- iv) Avoid changing API, routing or state logic
- v) Run Poused validation
- vi) Report exactly which file changed.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Month

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

Day

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

2. Add Dashboard Ticket Search.

Task 2 feature was very similar to the times.

- ① Dashboard 6 ticket search শাকর.
- ② Search and status filter লকজাশ কাজ করত.
- ③ Search কর কোন ticket পাওয়া না গেল
new empty message দেখাত.
- ④ status filter এর কারনে কোন ticket না থাকলে
আবার empty message দে-ই শাকর.
- ⑤ দুই ধরনের empty state আলাদা রাখা হবে.

Year :

Diary

Subj :

Date →

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

1-1203

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

3. Enforce Ticket Lifecycle Transition.

Task-3 was noticeably easier to implement with Agent.md, mainly because it provided clear decision-making rules.

The final feature was not dramatically more complex with Agent.md. The code change was still small. The biggest improvement was it reduced uncertainty.

Without: The hardest part was deciding what the requirement meant.

with: The hardest part was mostly implementation and verification.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

2020

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

2. Implement Ticket Assignment and Agent Work Queues.

A Fullstack Feature where

- ① Create new relationship with Ticket.
- ② Create new migration for database.
- ③ Create protected endpoint so that only authorized user/agent can access it.
- ④ Adding role-based permission. Who can assign ticket who can see that etc.
- ⑤ Adding necessary filter
- ⑥ Adding necessary control in Dashboard UI for assigning ticket.

Agent যেন ticket নিজের কাছে assign করতে পারে and নিজের assign করা ticket সুলভিত আলাদা work queue দেখতে পারে। এমন frontend + backend feature create করা

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

MON

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

D
A
Y

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

5. Harden Authentication and session Management

① JWT validation strong करा - token सचि
उ valid कि ता check करा

② Centralized role dependencies create करा.
User एव role अनुसार permission check
करा एकटि केलीय बाबता.

③ Startup Configuration validation add करा.
Application start होयार मय important
configuration/security settings सचि वाड
कि ता check करा

④ Frontend session handling Improve करा.
Frontend उ login session token perfectly
manage करा.

Full application उ authentication and login session
उ security सकियाल करा, सत unauthorized
user access करा ता भाव and session-related
देन करा.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

MONTH

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

Outputs were compared against a rubric fixed in advance scoring.

- i) Correctness against the stated requirement.
- ii) Adherence to existing project convention.
- iii) presence of unrequested or out-of-scope changes.
- iv) Handling of edge cases and failure paths.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

2022

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

Task-3

With agent.md thoes business policy is already been declared in the Agent.md file, so the task become implementation and verification rather than policy invention.

Task-4

In one sentence: On this codebase, the context file did not make the agent smarter it made the agent's unsupervised guesses fewer and on the two task large enough for guesses to compound, that difference was the entire story.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

MONTH

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

① Unidesk is a full-stack internal IT support ticketing system.

How ② Frontend is react which communicate with a FastAPI backend.

③ The backend uses SQL Alchemy to interact with PostgreSQL.

④ Pydantic handles request validation.

⑤ JWT handles authentication.

⑥ Role-based authorization controls employee and support agent action.

⑦ Alembic manages database migration, and pytest is used for backend testing.

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Month

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

Day

☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

Backend API level must attack test

1. Missing Input Attack
2. Empty Input
3. Null Input
4. Whitespace Input
5. Invalid data type
6. Boundary value
7. Very Large Input
8. Special character
9. Duplicate Data
10. Invalid Format
11. Invalid Id
12. Non-existing Resource
13. Unauthorized Access
14. Authentication Bypass
15. Authorization / RBAC
16. Token Manipulation
17. Expired Token
18. Wrong credentials
19. SQL Injection
20. XSS Attack
21. Parameter Tampering
22. HTTP Method Manipulation
23. Rate Limit / Brute Force Attack
24. Concurrent Request Attack
25. Error Handling

Year :

Diary

Subj :

Date → 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

2023

☐ January ☐ February ☐ March ☐ April ☐ May ☐ June ☐ July
☐ August ☐ September ☐ October ☐ November ☐ December

DAY
☐ Saturday ☐ Sunday ☐ Monday ☐ Tuesday
☐ Wednesday ☐ Thursday ☐ Friday

26. Unexpected API Response
27. File upload
28. Session Handling
29. Data Leakage
30. CORS/Security Headers Attack.

Not biendly, I would verify the result agent's ~~test output~~ logs, traceback and the actual system behavior.